

Cloud Computing Lab 3: Docker

Due Date

Monday, September 29, 2025 11:59 PM

Objectives

In this lab, you will learn about Docker and how to “dockerize” programs you’ve created for fast shipping and deployment.

What is Docker?

Docker is a software that allows for the building, testing, and deployment of applications at a high rate. It does so by packaging software into units called ‘containers’, which practically work like a mini virtual machine, that include the runtime, libraries, code, and anything else needed to run an application.

Setting up Docker

For this lab, we’ll be using the Docker CLI (command line interface) that allows us to use docker commands from the command line/terminal. There is no completely direct way to only download the CLI, however, so we’ll be downloading Docker Desktop. Follow the instructions for whichever system you are running on. From here, you should have access to docker commands. To be sure, run this to check your version.

```
1 > docker --version
```

You should also set up a Docker Hub account. Docker Hub is an online library of public and private (given you have access) docker images that you can pull to your own device and run. We’ll be utilizing it to host our labs for quick deployment and testing. Create an account, and be sure to remember your account name as you’ll need it later.

Dockerfiles

You’re not going to be writing much code for this lab. Instead, you’ll be formatting your project to be pushed to Docker Hub and pulled wherever necessary. This can even be done on AWS, which is what we’ll have you do. You will be creating one new file, called a Dockerfile, that defines how your image sets up its own virtual environment. Here is an example Dockerfile, with comments to show what different parts of it do:

```
1 # Sets up the environment with a base of Python, specifically Python 3
2 FROM python:3
3
4 # Copies all files from the directory this file is placed in,
5 # into the environment being created
6 COPY . .
7
8 # Runs the command to install all modules in requirements.txt
9 # (which was just copied by the previous line)
10 RUN pip install -r requirements.txt
11
12 # Final command that activates the project
13 CMD ["py", "test.py"]
```

Your Dockerfile might look similar to this, and that is completely okay. What is important is understanding where different commands go and how they work together. The trickiest part of this file is most likely going to be the CMD and what to put in it, but I'm sure you'll be able to figure it out!

Environment Variables

If you were to build and run your project locally now, you're going to run into an issue with authentication. Now that your application isn't hosted on EC2, your code is trying to access an S3 bucket without any sort of authentication. When its run on your EC2 instance, the IAM group we made in Lab 2 is what allows us to bypass any sort of access keys or logins when it comes to using S3. The fix for this, then, is to start logging in directly in the code. However, one can imagine that directly typing your username and password into your code and keeping it there, unencrypted and completely unsafe, is not the best way to go about things.

Instead, then, we'll be using environment variables. These are kept in a file that everyone who uses your program should have their own copy of on their own computer. Sometimes it has the exact same variables as someone else, or could be different depending on users.

The way we configure these is to have a file called **envVariables**, which is a line-by-line list of variables:

```
1 FAVORITE_COLOR=red
2 PERSONAL_ACCESS_TOKEN=abcd1234
3 SUPER_SECRET_PASSWORD=password
```

The great part about Python is that it can recognize these environment variables when loaded in. Typically this file is a hidden file named **.env**. We would also use a package called **python-dotenv** that allows us to load in environment variables directly from that hidden **.env**. Docker allows us to specify a file to use as the environment variables, so we don't package **.env** in the image itself (which would obviously not be ideal). In this course, we will avoid using the hidden file and instead create a file named **envVariables** and place it outside of the folder containing our Dockerfile. It is very important to place this file outside of our Docker image, as we will place this Docker image on Docker Hub and we do not want to share the information in our environment variables.

Using environment variables requires two steps. First, we need to edit our code to allow for accepting these environment variables, as well as authenticating the S3 client. Then, we'll need to be sure that when we run the Docker image, we link it with the **envVariables** file.

I will not give you the code to authenticate in Python directly, however, here is an example of how to get environment variables in your code:

```
1 # import necessary to get environment variables
2 import os
3
4 # favorite color from my .env
5 # notice the key being the same capitalization as in the file
6 favorite_color = os.environ["FAVORITE_COLOR"]
7
8 print("My favorite color is", favorite_color)
9 # ^^ will print "My favorite color is red"
```

You can edit your code in Lab 1/2 to include an access key and secret key, which will authenticate you to use the S3 bucket. You can create these keys using methods described here. You'd change the method in your code to look like this:

```
1 s3 = boto3.resource("s3",
2     aws_access_key_id=<your access key environment variable>,
3     aws_secret_access_key=<your secret access key env var>)
```

Finally, when you run your docker image (which you'll figure out yourself), you're going to need to use the flag `--env-file` to specify the environment file, like so (if the terminal was in the same directory as `envVariables`):

```
1 > docker run --env-file ./envVariables <other stuff, no spoilers :) >
```

DIY

From here, it's time to build your project and run it and push it to Docker Hub. These are all done via the Docker CLI, of which here is a cheat sheet. What we want you to do is to build your API and run it on your EC2 instance. There may be complications when working with Docker on your EC2 instance, but remember Google and StackOverflow are your friends!

Hints and Resources

- When building your image, make sure to use a tag so you can find it on docker hub. It should look like `<docker-username>/<project-name>`
- When running your image, you'll need to use some arguments. Maybe something to specify ports...or to run it in the background...?
- Make sure that the ec2 instance running on port 8001, the Docker container will still listen on port 8000.
- Note to Mac users: If you build your Docker image on your local machine, it might have trouble running on EC2 instances due to architecture differences. If you encounter issues, consider building your image directly on the EC2 instance or use the `-platform` flag when building to specify the target platform (e.g., `-platform linux/amd64`).

Grading

- 5 pts - Create Dockerfile
- 5 pts - Create Environment Variables to access S3 Bucket
- 5 pts - Push Docker Image to Docker Hub
- 45 pts - Deploy Movies API Docker Instance on AWS

Submission

Submit this assignment to your lab 3 repo on GitHub before the due date. Be sure to include all relevant project files in your submission. Also, make sure to include a link to your EC2 instance running on port 8001 and your Docker Hub repo name in the README.md.